Project

Ch no 5:

Q1: (Find the Smallest Integer) Write a program that uses a for statement to find the smallest of several integers. Assume that the first value read specifies the number of values remaining.

```
#include <iostream>

using namespace std;


int main() {
   int n;
   cout << "Enter how many numbers: ";
   cin >> n;

   int num, smallest;
   cin >> smallest;   // first number

   for (int i = 1; i < n; i++) {
      cin >> num;
      if (num < smallest) {
         smallest = num;
      }
   }
```

```
    cout << "Smallest number is: " << smallest << endl;
    return 0;
}
```

Q2: (Product of Odd Integers) Write a program that uses a statement to calculate and print the product of the odd integers from 1 to 15.

```
#include <iostream>
using namespace std;

int main() {
    long long product = 1;

    for (int i = 1; i <= 15; i += 2) {
        product *= i;
    }

    cout << "Product of odd numbers from 1 to 15 is: " << product << endl;
    return 0;
}
```

Q3: (Factorials) The factorial function is used frequently in probability problems. Using the definition of factorial in Exercise 4.34, write a program that uses a statement to evaluate the factorials of the integers from 1 to 5. Print the results in a tabular format. What difficulty might prevent you from calculating the factorial of 20?

```
#include <iostream>
using namespace std;
```

```
int main() {
   cout << "Number\tFactorial" << endl;
   cout << "--------------------" << endl;

   for (int i = 1; i <= 5; i++) {
      long long factorial = 1; // Use long long for larger factorials
      for (int j = 1; j <= i; j++) {
         factorial *= j;
      }
      cout << i << "\t" << factorial << endl;
   }

   return 0;
}
```

Output:

```
Number Factorial
--------------------
1      1
2      2
3      6
4      24
5      120
```

Q4: (Compound Interest) Modify the compound interest program of Section 5.4 to repeat its steps for the interest rates 5%, 6%, 7%, 8%, 9% and 10%. Use a for statement to vary the interest rate.

```
#include <iostream>

#include <iomanip>

using namespace std;


int main() {

   double principal = 1000.0;


   cout << "Rate\tAmount after 10 years" << endl;


   for (int rate = 5; rate <= 10; rate++) {

      double amount = principal;

      for (int year = 1; year <= 10; year++) {

         amount += amount * rate / 100;

      }

      cout << rate << "%\t" << fixed << setprecision(2) << amount << endl;

   }


   return 0;

}
```

Q5: (Drawing Patterns with Nested for Loops) Write a program that uses for statements to print the following patterns separately, one below the other.

Use for loops to generate the patterns. All asterisks (*) should be printed by a single statement of the form cout << '*'; (this causes the asterisks to print side by side). [Hint: The last two patterns require that each line begin with an appropriate number of blanks. Extra credit: Combine your code from the four separate problems into a single program that prints all four patterns side by side by making clever use of nested for loops.]

(a)

```
#include <iostream>

using namespace std;


int main() {
   for (int i = 1; i <= 10; i++) {
      for (int j = 1; j <= i; j++) {
         cout << "*";
      }
      cout << endl;
   }
   return 0;
}
```

Output:

```
*
**
***
****
```

*****

******

****

****

****

****

(b)

```
#include <iostream>
using namespace std;

int main() {
   for (int i = 10; i >= 1; i--) {
      for (int j = 1; j <= i; j++) {
         cout << "*";
      }
      cout << endl;
   }
   return 0;
}
```

Output:

****

****

****

******

*****

****

***

**

*

(c)

```
#include <iostream>
using namespace std;

int main() {
   for (int i = 10; i >= 1; i--) {
      for (int space = 1; space <= 10 - i; space++) {
          cout << " ";
      }
      for (int star = 1; star <= i; star++) {
         cout << "*";
      }
      cout << endl;
   }
   return 0;
}
```

Output:

```
**********
 *********
  *******
   ******
    *****
     ****
      ***
```

```
         **
          *
```

(d)

```
#include <iostream>
using namespace std;

int main() {
   for (int i = 1; i <= 10; i++) {
      for (int space = 1; space <= 10 - i; space++) {
         cout << " ";
      }
      for (int star = 1; star <= i; star++) {
         cout << "*";
      }
      cout << endl;
   }
   return 0;
}
```

Output:

```
           *
          **
         ***
        ****
```

```
     *****
    ******
   *******
  ********
 *********
**********
```

Q6:

(Calculating π) Calculate the value of π from the infinite

Print a table that shows the approximate value of π after each of the first 1000 terms of this series.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
   double pi = 0.0;
   int denominator = 1;
   int sign = 1;

   cout << setw(10) << "Term" << setw(20) << "Approximation of Pi" << endl;
   cout << "-----------------------------------------" << endl;

   for (int i = 1; i <= 1000; i++) {
      pi += sign * (4.0 / denominator);
      denominator += 2;
```

```
    sign *= -1;


    cout << setw(10) << i
        << setw(20) << fixed << setprecision(10) << pi << endl;
  }


  return 0;
}
```

Q6: Diamond of Asterisks) Write a program that prints the following diamond shape. You may use output statements that print a single asterisk (*), a single blank or a single newline. Maximize your use of repetition (with nested for statements) and minimize the number of output statements.

```
#include <iostream>

using namespace std;


int main() {
  int n = 5;   // half height of diamond


  // Upper half
  for (int i = 1; i <= n; i++) {
    for (int s = 1; s <= n - i; s++)
      cout << " ";


    for (int star = 1; star <= 2 * i - 1; star++)
      cout << "*";
```

```
    cout << endl;
  }

  // Lower half
  for (int i = n - 1; i >= 1; i--) {
    for (int s = 1; s <= n - i; s++)
      cout << " ";

    for (int star = 1; star <= 2 * i - 1; star++)
      cout << "*";

    cout << endl;
  }

  return 0
}
```

Q7:

(Diamond of Asterisks) Modify Q 6 to read an odd number in the range 1 to 19 to specify the number of rows in the diamond, then display a diamond of the appropriate size.

```
#include <iostream>

using namespace std;
```

```
int main() {
    int rows;

    // Input
    cout << "Enter an odd number between 1 and 19: ";
    cin >> rows;

    // Validate input
    if (rows < 1 || rows > 19 || rows % 2 == 0) {
        cout << "Invalid input!" << endl;
        return 0;
    }

    int mid = rows / 2 + 1;

    // Upper half (including middle row)
    for (int i = 1; i <= mid; i++) {
        for (int s = 1; s <= mid - i; s++)
            cout << " ";

        for (int star = 1; star <= 2 * i - 1; star++)
            cout << "*";

        cout << endl;
```

```
    }

    // Lower half
    for (int i = mid - 1; i >= 1; i--) {
        for (int s = 1; s <= mid - i; s++)
            cout << " ";

        for (int star = 1; star <= 2 * i - 1; star++)
            cout << "*";

        cout << endl;
    }

    return 0;
}
```

CH no. 6:

Q1: (Find the Minimum) Write a program that inputs three double-precision, floating-point numbers and passes them to a function that returns the smallest number.

```
#include <iostream>
using namespace std;

// Function to find the smallest number
double smallest(double x, double y, double z) {
```

```
    double min = x;

    if (y < min)
        min = y;
    if (z < min)
        min = z;

    return min;
}

int main() {
    double a, b, c;

    // Input
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;

    // Function call and output
    cout << "The smallest number is: "
         << smallest(a, b, c) << endl;

    return 0;
}
```

Q2: Perfect Numbers) An integer is said to be a perfect number if the sum of its divisors, including 1 (but not the number itself), is equal to the number. For example, 6 is a perfect number, because 6 = 1 + 2 + 3. Write a function isPerfect that determines whether parameter number is a perfect number. Use this function in a program that determines and prints all the perfect numbers between 1 and 1000. Print the divisors of each perfect number to confirm that the number is indeed perfect. Challenge the power of your computer by testing numbers much larger than 1000.

```
#include <iostream>

using namespace std;


// Function to check whether a number is perfect

bool isPerfect(int number) {

   int sum = 0;


   for (int i = 1; i <= number / 2; i++) {

      if (number % i == 0)

         sum += i;

   }


   return (sum == number);

}


int main() {


   cout << "Perfect numbers between 1 and 1000:\n\n";
```

```
    for (int num = 1; num <= 1000; num++) {
        if (isPerfect(num)) {
            cout << num << " = ";

            // Print divisors
            for (int i = 1; i <= num / 2; i++) {
                if (num % i == 0)
                    cout << i << " ";
            }

            cout << endl;
        }
    }

    return 0;
}
```

Output:

Perfect numbers between 1 and 1000:

6 = 1 2 3

28 = 1 2 4 7 14

496 = 1 2 4 8 16 31 62

Q3: (Reverse Digits) Write a function that takes an integer value and returns the number with its digits reversed. For example, given the number 7631, the function should return 1367.

```
#include <iostream>

using namespace std;


// Function to reverse digits of a number

int reverseDigits(int number) {

   int reversed = 0;


   while (number != 0) {

      reversed = reversed * 10 + (number % 10);

      number /= 10;

   }


   return reversed;

}


int main() {

   int num;


   // Input

   cout << "Enter an integer: ";

   cin >> num;
```

```
    // Output
    cout << "Reversed number: " << reverseDigits(num) << endl;

    return 0;
}
```

Q4: (Greatest Common Divisor) The greatest common divisor (GCD) of two integers is the largest integer that evenly divides each of the numbers. Write a function gcd that returns the greatest common divisor of two integers.

```
#include <iostream>
using namespace std;

// Function to find GCD using Euclid's algorithm
int gcd(int a, int b) {
    while (b != 0) {
        int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

int main() {
    int x, y;
```

```
    // Input
    cout << "Enter two integers: ";
    cin >> x >> y;

    // Output
    cout << "Greatest Common Divisor: " << gcd(x, y) << endl;

    return 0;
}
```

Output:

Greatest Common Divisor: 12

Q5: (Quality Points for Numeric Grades) Write a function qualityPoints that inputs a student's average and returns 4 if a student's average is 90–100, 3 if the average is 80–89, 2 if the average is 70–79, 1 if the average is 60–69 and 0 if the average is lower than 60.

```
#include <iostream>
using namespace std;

int qualityPoints(int average) {
    if (average >= 90 && average <= 100)
        return 4;
    else if (average >= 80 && average <= 89)
        return 3;
```

```
    else if (average >= 70 && average <= 79)
        return 2;
    else if (average >= 60 && average <= 69)
        return 1;
    else
        return 0;
}

int main() {
    int avg;
    cout << "Enter student average: ";
    cin >> avg;

    int points = qualityPoints(avg);
    cout << "Quality points: " << points << endl;

    return 0;
}
```

Q6: (Recursive main) Can main be called recursively on your system? Write a program containing a function main. Include static local variable count and initialize it to 1. Postincrement and print the value of count each time main is called. Compile your program. What happens.

```
#include <iostream>
using namespace std;
```

```
int main() {
   static int count = 1;  // static variable, initialized only once

   cout << "Count: " << count << endl;
   count++;  // post-increment

   if (count <= 5) // limit recursion to avoid infinite loop / crash
      main();     // recursive call

   return 0;
}
```

Q7: Distance Between Points) Write function distance that calculates the distance between two points (x1, y1) and (x2, y2). All numbers and return values should be of type double.

```
#include <iostream>
#include <cmath> // for sqrt() and pow()
using namespace std;

// Function to calculate distance
double distance(double x1, double y1, double x2, double y2) {
   return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}
```

```
int main() {
   double x1, y1, x2, y2;

   cout << "Enter coordinates of first point (x1 y1): ";
   cin >> x1 >> y1;

   cout << "Enter coordinates of second point (x2 y2): ";
   cin >> x2 >> y2;

   double dist = distance(x1, y1, x2, y2);
   cout << "Distance between points: " << dist << endl;

   return 0;
}
```

Q8: (Circle Area) Write a C++ program that prompts the user for the radius of a circle, then calls inline function circleArea to calculate the area of that circle.

#include using namespace std;

// Inline function to calculate circle area inline double circleArea(double radius) { const double PI = 3.141592653589793; return PI * radius * radius; }

int main() { double radius;

```
cout << "Enter radius of the circle: ";
cin >> radius;

double area = circleArea(radius);
cout << "Area of the circle: " << area << endl;
```

```
return 0;

}
```

CH no.7:

Q1: (Bubble Sort) In the bubble sort algorithm, smaller values gradually "bubble" their way upward to the top of the array like air bubbles rising in water, while the larger values sink to the bottom. The bubble sort makes several passes through the array. On each pass, successive pairs of elements are compared. If a pair is in increasing order (or the values are identical), we leave the values as they are. If a pair is in decreasing order, their values are swapped in the array. Write a program that sorts an array of 10 integers using bubble sort.

```
#include <iostream>

using namespace std;


int main() {
   int arr[10];


   // Input 10 integers
   cout << "Enter 10 integers: ";
   for (int i = 0; i < 10; i++) {
      cin >> arr[i];
   }


   // Bubble Sort Algorithm
```

```
  for (int i = 0; i < 9; i++) {        // Number of passes
    for (int j = 0; j < 9 - i; j++) {  // Compare adjacent elements
      if (arr[j] > arr[j + 1]) {    // If out of order, swap
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }

  // Output sorted array
  cout << "Sorted array: ";
  for (int i = 0; i < 10; i++) {
    cout << arr[i] << " ";
  }
  cout << endl;

  return 0;
}
```

Q2: (Palindromes) A palindrome is a string that is spelled the same way forward and backward. Examples of palindromes include “radar” and “able was i ere i saw elba.” Write a recursive function testPalindrome that returns true if a string is a palindrome, and false otherwise. Note that like an array, the square brackets ([]) operator can be used to iterate through the characters in a string.

```
#include <iostream>
#include <string>
using namespace std;

// Recursive function to check palindrome
bool testPalindrome(const string &str, int start, int end) {
   if (start >= end) // Base case: crossed the middle
      return true;
   if (str[start] != str[end]) // Characters don't match
      return false;
   return testPalindrome(str, start + 1, end - 1); // Recurse inward
}

int main() {
   string s;
   cout << "Enter a string: ";
   getline(cin, s); // Use getline to include spaces

   if (testPalindrome(s, 0, s.length() - 1))
      cout << "The string is a palindrome." << endl;
   else
      cout << "The string is not a palindrome." << endl;

   return 0;
```

}

Q3: (Linear Search) Modify the program in Fig. 7.18 to use recursive function linearSearch to perform a linear search of the array. The function should receive an integer array and the size of the array as arguments. If the search key is found, return the array subscript; otherwise, return –1.

```
#include <iostream>

using namespace std;


// Recursive linear search function

int linearSearch(int arr[], int size, int key, int index = 0) {

   if (index >= size)       // Base case: reached end of array

      return -1;          // Not found

   if (arr[index] == key)    // Key found

      return index;

   return linearSearch(arr, size, key, index + 1); // Check next element

}


int main() {

   int arr[] = {5, 3, 8, 6, 2, 9, 1, 7, 4, 0};

   int size = sizeof(arr) / sizeof(arr[0]);

   int key;


   cout << "Enter value to search: ";

   cin >> key;
```

```
    int result = linearSearch(arr, size, key);
    if (result != -1)
        cout << "Value found at index: " << result << endl;
    else
        cout << "Value not found in the array." << endl;

    return 0;
}
```

Q4: (Print an Array) Write a recursive function printArray that takes an array, a starting subscript and an ending subscript as arguments, returns nothing and prints the array. The function should stop processing and return when the starting subscript equals the ending subscript.

```
#include <iostream>
using namespace std;

// Recursive function to print array
void printArray(int arr[], int start, int end) {
    if (start == end) // Base case: stop when start reaches end
        return;

    cout << arr[start] << " ";
    printArray(arr, start + 1, end); // Recursive call for next element
}
```

```
int main() {
   int arr[] = {5, 10, 15, 20, 25};
   int size = sizeof(arr) / sizeof(arr[0]);

   cout << "Array elements: ";
   printArray(arr, 0, size); // start = 0, end = size
   cout << endl;

   return 0;
}
```

Q5: (Print a String Backward) Write a recursive function stringReverse that takes a string and a starting subscript as arguments, prints the string backward and returns nothing. The function should stop processing and return when the end of the string is encountered. Note that like an array the square brackets ([]) operator can be used to iterate through the characters in a string.

```
#include <iostream>
#include <string>
using namespace std;

// Recursive function to print string backward
void stringReverse(const string &str, int index) {
   if (index >= str.length()) // Base case: reached the end
      return;
```

```
    // Recursive call first
    stringReverse(str, index + 1);

    // Print character after recursion
    cout << str[index];
}

int main() {
    string s;
    cout << "Enter a string: ";
    getline(cin, s);

    cout << "Reversed string: ";
    stringReverse(s, 0); // start from index 0
    cout << endl;

    return 0;
}
```

Q6: (Find the Minimum Value in an Array) Write a recursive function recursiveMinimum that takes an integer array, a starting subscript and an ending subscript as arguments, and returns the smallest element of the array. The function should stop processing and return when the starting subscript equals the ending subscript.

```
#include <iostream>
```

```
using namespace std;

// Recursive function to find minimum in an array
int recursiveMinimum(int arr[], int start, int end) {
    if (start == end) // Base case: only one element
        return arr[start];

    // Recursive call to find minimum in the rest of the array
    int minRest = recursiveMinimum(arr, start + 1, end);

    // Return the smaller of current element and minRest
    return (arr[start] < minRest) ? arr[start] : minRest;
}

int main() {
    int arr[] = {12, 5, 7, 3, 9, 2, 8};
    int size = sizeof(arr) / sizeof(arr[0]);

    int minValue = recursiveMinimum(arr, 0, size - 1);
    cout << "Minimum value in the array: " << minValue << endl;

    return 0;
}
```